WEEK #7

1# A cryptographic security module uses "Narcissistic Numbers" (Armstrong numbers) as a part of a checksum validation. Write a Python program using a while loop to check if a user-provided numeric key is an Armstrong number. Example 1: • Input: 153 • Output: Armstrong number Example 2: • Input: 9474 • Output: Armstrong number

```
def armstrong(n):
    digits = len(str(n))
    temp = n
    total = 0
    while temp > 0:
        u = temp % 10
        total += (u**digits)
        temp //= 10
    return total == n
n = int(input("Enter numeric key value: "))
if armstrong(n):
    print("Armstrong Number")
else:
    print("Not an Armstrong Number")
```

```
Enter numeric key value: 9474
Armstrong Number
```

2# A biological simulation tool models the growth of a cell culture following the Fibonacci sequence. Write a recursive python function to generate the first n terms of the sequence to predict population growth steps

```
def fib(n):
    if n<=0:
        return 0
    if n==1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

```
n = int(input("Enter value of n: "))

for i in range(n+1):

    print(fib(i),end=" ")
```

```
Enter value of n: 6
0 1 1 2 3 5 8
```

3# A data science team is building a module to calculate permutations (combinations of arrangements) for a logistics algorithm. To compute the number of possible routes, they need a core component that calculates factorials efficiently. Write a recursive Python function named calculate_routes(n) that accepts a positive integer n and returns its factorial (the product of all positive integers less than or equal to n). Example: Input: n=5 → Output: 120

```
def calculate_routes(n):

    if n<=1:

        return 1

    else:

        return n * calculate_routes(n-1)


n= int(input("Enter the number n: "))

print(f"Factorial of {n} = {calculate_routes(n)}")
```

```
Enter the number n: 5
Factorial of 5 = 120
```

4# A telecommunications engineer is designing a system to synchronize two signals with different frequencies. To find the moment when both signals align perfectly, the system needs to determine the greatest common divisor (GCD) of their cycle times. Write a recursive Python function named find_signal_sync(a, b) that takes two integers a and b (representing the cycle times) and returns their Greatest Common Divisor using the Euclidean algorithm. Example: Input: a=48, b=18 → Output: 6

```
def find_signal_sync(a, b):

    if b==0:

        return a

    else:

        return find_signal_sync(b,a%b)


a=int(input("Enter cycle time a: "))
```

```
b=int(input("Enter cycle time b: "))

print(find_signal_sync(a,b))
```

```
Enter cycle time a: 88
Enter cycle time b: 120
8
```

5# Write a program in Python to print the following Pattern. (for)

```
* * * *

* * *

* *

*
```

```
n = 4

for i in range(n, 0, -1):

    for j in range(i):

        print("*", end=" ")

    print()
```

```
* * * *
* * *
* *
*
```

6# A network packet analyzer receives a batch of data packet IDs (integers). Write a Python program to filter these IDs, placing even IDs into one list (low priority) and odd IDs into another list (high priority).

```
l=eval(input("Enter the data packet IDs: "))

even = []

odd = []


for i in l:

    if i%2==0:

        even.append(i)

    else:

        odd.append(i)
```

```
print(even)

print(odd)
```

```
Enter the data packet IDs: 1,2,3,4,5,6
[2, 4, 6]
[1, 3, 5]
```

7# Given an integer x, return true if x is a palindrome, and false otherwise. Example: Input: x = 121 Output: true Explanation: 121 reads as 121 from left to right and from right to left.

```
x =int(input("Enter value of x: "))

rev = 0

temp = x

place = 1

while temp>0:

    d = temp % 10

    rev = rev +d*place

    temp //=10

    place *= 10


if x == rev:

    print(True)

else:

    print(False)
```

```
Enter value of x: 12021
True
```

8# An inventory management system is consolidating records from two different warehouses. Write a Python program to merge two lists of product IDs and display the final list in sorted order. Example 1: • Input: [1,2], [3,4,5] • Output: [1,2,3,4,5] Example 2: • Input: [1,2], [2,3] • Output: [1,2,2,3]

```
list1 = list(map(int, input("Enter first list elements: ").split()))

list2 = list(map(int, input("Enter second list elements: ").split()))

merged = list1 + list2

merged.sort()

print("Sorted merged list:", merged)
```

```
Enter first list elements: 1 2 4
Enter second list elements: 6 7 5
Sorted merged list: [1, 2, 4, 5, 6, 7]
```

9# An algorithm visualization tool demonstrates how different sorting logic works. Write a Python program to find the second largest number in a list. (use Bubble Sort algorithm)

```
numbers = list(map(int, input("Enter numbers: ").split()))

n = len(numbers)

# Bubble Sort

for i in range(n):

    for j in range(0, n-i-1):

        if numbers[j] > numbers[j+1]:

            numbers[j], numbers[j+1] = numbers[j+1], numbers[j]


largest = numbers[-1]


for i in range(n-2, -1, -1):

    if numbers[i] != largest:

        print("Second Largest:", numbers[i])

        break
```

```
Enter numbers: 1 2 3 4 4 3 5
Second Largest: 4
```